Software Supply Chain Attacks and Build Provenance

A Paper-Style Report on npm Ecosystem Risks and Provenance-Based Defense

NG, Chun Sing (r14922186) OU, Chia-Yun (r14922072)

May 23, 2026

Abstract

Modern software development depends heavily on open-source packages, package managers, continuous integration and continuous deployment pipelines, and third-party package registries. These mechanisms improve development efficiency, but they also expand the attack surface of modern software systems. A software supply chain attack does not need to attack the final application directly. Instead, an attacker can compromise a trusted dependency, maintainer account, build cache, CI/CD workflow, package registry, or release pipeline, and then use the existing trust relationship to deliver malicious code to downstream users.

This report studies software supply chain attacks with a focus on the npm ecosystem and build provenance. First, it explains why software supply chain failures have become a major risk in modern application security. Second, it analyzes two recent case studies: the Axios npm compromise and the TanStack npm supply-chain compromise. The Axios incident illustrates how stolen maintainer credentials and malicious installation scripts can silently install malware during normal package installation. The TanStack incident reveals a deeper limitation: even valid build provenance may be insufficient if the legitimate CI/CD workflow itself has been compromised.

The central argument of this report is that build provenance is necessary but not sufficient. Provenance can improve artifact traceability, make tampering more visible, and enable install-time verification. However, provenance alone cannot prove that an artifact is benign. Effective software supply chain security requires a layered defense strategy that

combines dependency scanning, lockfile discipline, install-time verification, maintainer account security, CI/CD isolation, cache hardening, credential restriction, and provenance verification.

Keywords: software supply chain security, build provenance, SLSA, Sigstore, npm, CI/CD security, dependency attack

Table of Contents

1 Introduction

Modern software development is no longer a fully local or fully controlled process. Developers routinely execute commands such as the following:

```
npm install
pip install
mvn install
conda install
```

These commands automatically download third-party code, resolve transitive dependencies, and, in some ecosystems, execute installation lifecycle scripts. From an engineering perspective, this automation greatly improves productivity because developers can reuse existing software components instead of implementing everything from scratch. From a security perspective, however, it means that developers delegate a significant amount of trust to package maintainers, package registries, CI/CD pipelines, build caches, release automation, and intermediate tooling.

The central danger of software supply chain attacks is that attackers do not need to break into the final application directly. They can instead compromise an upstream component or an intermediate process that is already trusted by the developer or organization. Once a widely used dependency is compromised, malicious code may be automatically downloaded, built, or executed by many downstream projects. This gives software supply chain attacks a high amplification effect and makes them difficult to handle using traditional application-boundary security models.

This report focuses on the following research question: **Can build provenance effectively prevent software supply chain attacks?** The answer is nuanced. Build provenance can raise the cost of attack and strengthen artifact verification, but it is not a complete solution. If the CI/CD pipeline, build cache, or workflow permissions are compromised, provenance may still produce a valid-looking record for a malicious artifact. Therefore, provenance must be used together with a broader defense strategy.

2 Background: Software Supply Chain Security

2.1 Definition of Software Supply Chain

According to NIST, a software supply chain is a collection of steps that create, transform, and assess the quality and policy conformance of software artifacts. In practice, the software supply chain includes not only source code, but also third-party dependencies, version control systems, build tools, CI/CD pipelines, package registries, release workflows, and deployment mechanisms.

A software supply chain attack can therefore be understood as an attack in which an adversary compromises or manipulates a trusted node in this chain. The compromised node may be a dependency package, maintainer account, build cache, CI/CD workflow, artifact registry, signing mechanism, or publishing process. Instead of exploiting a vulnerability in the final application, the attacker abuses the trust relationship between developers, tools, packages, and registries.

2.2 Why This Risk Has Become Critical

OWASP Top 10:2025 ranks Software Supply Chain Failures as A03, reflecting a shift in modern application security risk. Traditional application security often focuses on flaws inside the application itself, such as injection, broken authentication, insecure design, or access control failures. Modern software, however, depends on a much larger ecosystem. Security now also depends on how the application is built, which packages it depends on, who publishes those packages, and whether the final artifact can be verified.

Several factors make this risk increasingly important:

(1) **Large dependency graphs:** A project may declare only a small number of direct dependencies, but those dependencies can introduce many transitive packages.

(2) **Highly automated installation workflows:** Package managers automatically resolve versions, download packages, read metadata, and may execute lifecycle scripts.

(3) **CI/CD as the release backbone:** Many packages are built and published automatically by GitHub Actions or other CI/CD systems, mak-

ing workflow permissions and runner environments important attack surfaces.

(4) **Limited human inspection:** Developers usually do not manually inspect every package version, tarball, build log, artifact hash, and provenance record.

(5) **Large downstream impact:** If a popular upstream package is compromised, the impact can spread across many organizations and downstream products.

3 Threat Model and Attack Surfaces

Software supply chain attacks can be analyzed across four layers: the dependency layer, the source layer, the build layer, and the release layer. Each layer contains different attack surfaces and requires different defenses.

Table 1: Major attack surfaces in the software supply chain

Layer	Possible Attack Surface
Dependency	Malicious packages, typosquatting, dependency confusion, transitive dependency compromise, abuse of preinstall/install/postinstall scripts
Source	Compromised maintainer accounts, malicious commits, review bypass, weak protected branch settings
Build	CI/CD workflow tampering, build cache poisoning, token extraction from runner memory, unsafe pull request workflows
Release	Registry account compromise, artifact substitution, missing signatures, missing provenance verification

In the npm ecosystem, lifecycle scripts are especially important. Scripts such as `preinstall`, `install`, and `postinstall` can run automatically during package installation. If an attacker publishes a package version containing a malicious lifecycle script, the victim may be compromised during installation even before the imported package is used in application code.

4 Case Study I: Axios npm Compromise

4.1 Incident Overview

Axios is a widely used HTTP client in the JavaScript ecosystem. According to the source material, Axios suffered an npm supply chain compromise on March 31, 2026. The attacker used stolen maintainer credentials to publish malicious versions of the package. These malicious versions introduced an attacker-controlled dependency and used an installation script to install a remote access trojan (RAT) during normal package installation.

This incident is representative because it shows a common risk in open-source package ecosystems. When the publishing authority of a popular package is compromised, downstream users may not immediately notice anything suspicious. The package name, registry page, and versioning process can still appear normal, even though the actual tarball has been modified by the attacker.

4.2 Attack Flow

The Axios incident can be conceptualized as the following chain:

```
npm install axios
  -> malicious axios version
      -> attacker-controlled dependency
          -> postinstall script executes
              -> RAT installed silently
```

The dangerous aspect of this attack is that it follows a normal developer workflow. A user may simply run `npm install`, but the installation process silently executes malicious code. In this sense, the compromise happens before the application even runs. The installation phase itself becomes the execution vector.

4.3 Security Gaps Revealed by the Case

The Axios incident demonstrates several limitations in common defenses:

(1) **Time lag in CVE-based scanning:** Tools such as `npm audit` and Snyk are useful for known CVEs, but a newly published malicious version may not yet have any CVE record.

(2) **Lockfiles are not absolute protection:** A lockfile can pin versions and hashes, but if the lockfile is generated while the malicious version is available, it may preserve the compromised version.

(3) **Automated update tools can introduce risk:** Tools such as Dependabot help update vulnerable dependencies, but an update may also introduce a newly compromised version.

(4) **Users lack install-time visibility:** Standard package installation does not usually require users to inspect package provenance, publisher identity, or lifecycle script behavior.

The Axios incident therefore supports an important conclusion: software supply chain defense cannot rely only on known-vulnerability scanning. It must also consider package metadata, publishing behavior, maintainer trust, provenance status, and install-time risk.

5 Build Provenance as a Defense Mechanism

5.1 Concept of Build Provenance

Build provenance is metadata that describes how a software artifact was produced. It typically records information such as the source repository, commit hash, builder identity, build workflow, build time, and artifact hash. Its purpose is to help consumers answer questions such as:

- Was this artifact built from the expected source commit?
- Was it produced by the expected CI/CD workflow?
- Were the build steps consistent with the expected build process?
- Does the tarball published to the registry match the CI output?

In the SLSA and Sigstore model, build provenance can be verified through keyless signing, OIDC identity, artifact hashes, and transparency logs. Ideally, it makes the path from source to release tamper-evident.

5.2 Producer and Consumer Workflow

A provenance-based workflow has two sides: producer and consumer.

Producer side:

(1) The CI/CD system builds the artifact from a specified commit in the source repository.

(2) The build process generates a provenance statement that records the commit, builder, build type, and artifact hash.

(3) The provenance is signed using a mechanism such as Sigstore and recorded in a transparency log.

(4) The registry stores the package together with its attestation metadata.

Consumer side:

(1) The consumer obtains the package and its corresponding attestation before installation.

(2) The consumer verifies the provenance signature.

(3) The consumer checks whether the source repository, builder identity, build type, and policy match expected values.

(4) If a package previously had provenance but the new version suddenly lacks provenance, the tool can warn or block the installation.

This model turns provenance from a passive record into a preventive control. Provenance itself is only metadata; the actual security benefit comes from install-time verification and policy enforcement.

5.3 Technology Stack

Table 2: Technology layers related to build provenance

Layer	Technology	Role
Framework	SLSA	Defines what provenance should prove and describes build levels
Cryptography	Sigstore	Provides keyless signing and transparency log support
Platform	Registry + CI	Generates, stores, and publishes attestations
Consumer	npq / Socket Firewall	Acts as a pre-install gate for provenance verification or package risk checks

The greatest value of build provenance is traceability. When an artifact is published to a registry, consumers no longer need to rely only on the package name and version number. They can require evidence about where the artifact came from and how it was built.

6 Case Study II: TanStack npm Supply-Chain Compromise

6.1 Incident Overview

The TanStack incident demonstrates the limitation of build provenance. According to the source material, TanStack experienced an npm supply-chain compromise on May 11, 2026. In a short period of time, 42 packages were affected and 84 malicious versions were published. The payload targeted cloud credentials, API tokens, and SSH keys.

Unlike the Axios case, the key issue was not simply the theft of an npm maintainer account followed by manual publication. The attack targeted the CI/CD workflow and cache trust boundary. A fork pull request was used to poison a shared CI cache. When a legitimate release workflow later ran, it restored the poisoned cache and produced malicious packages through the legitimate pipeline.

6.2 Why This Challenges Provenance

The most important lesson of the TanStack incident is that the malicious versions had valid build provenance. In other words, provenance correctly recorded that the artifacts came from the TanStack workflow. However, because the workflow environment had been compromised through cache poisoning and token extraction, the provenance record was valid but insufficient.

This leads to a central observation:

Provenance proves origin, not innocence.

Provenance can prove which repository, builder, and workflow produced an artifact, but it cannot by itself prove that the workflow was clean when it ran. Build provenance is therefore a necessary condition for stronger supply chain security, but it is not a sufficient condition.

6.3 CI/CD-Level Risks

The TanStack incident highlights several CI/CD-level risks:

(1) **Cache poisoning:** If an untrusted workflow and a trusted release workflow share a cache, an attacker may inject malicious content through the cache.

(2) **OIDC token exposure:** If tokens in a CI runner can be read by malicious code, the attacker may obtain publishing capability or abuse a legitimate workflow.

(3) **Unclear workflow trust boundaries:** Fork pull requests, `pull_request_target`, and release workflows become dangerous when privilege separation is weak.

(4) **Abuse of legitimate pipelines:** If the attacker can make the legitimate pipeline build and publish malicious artifacts, traditional provenance checks may still pass.

Therefore, build provenance must be combined with CI/CD hardening. Otherwise, provenance may provide an accurate but incomplete record.

7 Comparison of Defense Mechanisms

Table 3: Comparison of common software supply chain defenses

Defense	**Protects Against**	**Limitations**
`npm audit` / Snyk	Known CVEs and known vulnerable dependencies	May miss newly published malicious packages, unregistered CVEs, or abnormal registry behavior
Lockfile	Pins versions and hashes; improves reproducibility	If generated during a malicious exposure window, it may lock the compromised version

Defense	Protects Against	Limitations
Dependabot	Automatically updates known vulnerable dependencies	Still depends on vulnerability databases; an update may introduce a malicious version
Build provenance	Verifies artifact origin, builder identity, artifact hash, and build process metadata	Cannot by itself prove that the builder or CI/CD workflow was uncompromised
Install-time verification	Enforces provenance or package-risk policy before installation	Requires expected policies and depends on ecosystem adoption
CI/CD hardening	Reduces abuse of workflows, caches, tokens, and runners	Requires engineering effort and project-specific configuration

This comparison shows that no single defense covers the entire software supply chain attack surface. `npm audit` is useful for known vulnerabilities, but insufficient for zero-day malicious package publication. Lockfiles improve reproducibility, but do not determine whether a package is malicious. Build provenance improves source and build traceability, but cannot replace the security of the build environment. The most reasonable strategy is therefore defense in depth.

8 Toward a Layered Defense Strategy

Based on the above cases and analysis, a practical software supply chain defense strategy should cover at least four layers.

8.1 Dependency Layer

The goal of the dependency layer is to reduce the probability that malicious or vulnerable packages enter the project. Useful strategies include:

- Use lockfiles and avoid unnecessary dependency updates.
- Use SBOMs to record the dependency composition of the project.

- Integrate vulnerability scanners such as `npm audit`, OSV, and Snyk.
- Use install-time gates such as npq or Socket Firewall to check package metadata, provenance status, and suspicious behavior.
- Apply a minimum release age policy to avoid immediately installing very new and potentially high-risk versions.

8.2 Source Layer

The source layer focuses on protecting source repositories and maintainer accounts from abuse. Useful strategies include:

- Require maintainers to use multi-factor authentication.
- Use protected branches and mandatory code review.
- Restrict package publishing permissions so that no single account has excessive authority.
- Monitor abnormal commits, releases, and token usage.

8.3 Build Layer

The build layer is the key lesson from the TanStack incident. Useful strategies include:

- Strictly separate untrusted pull request workflows from trusted release workflows.
- Avoid sharing high-trust caches between fork pull request workflows and release workflows.
- Minimize the permissions of GitHub Actions tokens, OIDC tokens, and registry tokens.
- Prevent credential material from being exposed in runner memory or logs.
- Review cache restore paths, build scripts, and release scripts more carefully.

8.4 Release Layer

The release layer aims to make artifacts verifiable during publication and installation. Useful strategies include:

- Use build provenance and signed attestations.
- Record artifact hashes, builder identities, and source commits in a transparency log.
- Verify provenance at install time, rather than only generating provenance at publish time.
- Warn or block installation if a package that previously had provenance suddenly lacks provenance.

9 Discussion

9.1 Practical Value of Build Provenance

The value of build provenance should not be underestimated. It addresses a major weakness in traditional package ecosystems: users often cannot tell whether an artifact in a registry actually came from the expected source code and build process. Provenance creates a verifiable relationship between source, builder, and artifact, making artifact substitution, unauthorized publishing, and some maintainer-compromise scenarios easier to detect.

Provenance also enables policy enforcement. For example, an organization can require that installed packages must have valid provenance, or that artifacts must be produced only by a trusted repository, builder, and workflow. This shifts supply chain security from informal trust to machine-verifiable policy.

9.2 Fundamental Limitation of Build Provenance

However, the limitation of build provenance must also be clearly understood. Provenance is not a malware scanner, and it is not a complete proof of build environment integrity. If a legitimate workflow is controlled or poisoned by an attacker, provenance will still record that the artifact came from that workflow. This is the central lesson of the TanStack incident.

Therefore, the correct interpretation of build provenance is:

> Build provenance can make the supply chain tamper-evident, but it cannot make a compromised build trustworthy.

This means that build provenance must be combined with CI/CD hardening, credential isolation, cache control, and install-time behavioral checks.

9.3 Possible Implementation Direction

A practical project based on this topic could implement an npm supply chain scanner that combines several complementary techniques:

(1) Parse `package.json` and lockfiles to construct a dependency graph.

(2) Use `npm install --package-lock-only --ignore-scripts --audit=false` to generate or update a lockfile without executing lifecycle scripts.

(3) Integrate `npm audit` and OSV to detect known vulnerabilities.

(4) Integrate npq as an install-time risk checker for package metadata and provenance status.

(5) Detect lifecycle scripts such as `preinstall`, `install`, and `postinstall`.

(6) Produce an HTML or JSON report showing vulnerability counts, suspicious packages, provenance status, and risk scores.

Such an implementation would demonstrate the complementary nature of existing tools. `npm audit` and OSV focus mainly on known vulnerabilities. npq focuses more on install-time package risk. A custom scanner can add lifecycle script analysis, metadata anomaly checks, and report generation.

10 Conclusion

Software supply chain attacks are dangerous because they exploit the trust relationships that make modern software development efficient. Developers trust package managers, registries, dependency maintainers, CI/CD workflows, and automated release processes. Attackers exploit these trusted relationships to introduce malicious code into downstream systems while preserving the appearance of normal development activity.

The Axios incident shows how maintainer account compromise and malicious package publication can quickly affect many users. The TanStack incident goes further and shows that even build provenance may not prevent an attack if the CI/CD workflow or build cache is compromised. Together, these cases show that software supply chain security cannot depend on a single tool or concept.

Build provenance is an important advance in software supply chain security. It can answer whether an artifact came from the expected source, builder, and build process, and it provides a foundation for tamper-evident verification. However, it cannot by itself prove that the artifact is benign. Effective defense requires layered controls, including dependency scanning, lockfile discipline, install-time verification, maintainer account protection, CI/CD isolation, cache hardening, credential restriction, and provenance verification.

In summary, the proper value of build provenance is not that it guarantees safety. Its value is that it makes trust relationships in the software supply chain more visible and verifiable. Future software supply chain security should treat provenance as a foundational mechanism, but deploy it together with stronger build environment security and runtime policy enforcement.

References

[1] National Institute of Standards and Technology. *NIST SP 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf

[2] OWASP. *Software Supply Chain Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html

[3] OWASP. *A03:2025 Software Supply Chain Failures*. https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/

[4] OWASP. *OWASP Top 10:2025 Introduction and Methodology*. https://owasp.org/Top10/2025/0x00_2025-Introduction/

[5] SLSA. *Supply Chain Threats*. https://slsa.dev/spec/v1.0/threats-overview

[6] SLSA. *About SLSA*. https://slsa.dev/spec/v1.2/about

[7] SLSA. *Verifying Artifacts*. https://slsa.dev/spec/v1.0/verifying-artifacts

[8] OpenSSF. *Sigstore*. https://www.sigstore.dev/

[9] Liran Tal. *npq: Safely install packages with npm or yarn by auditing them as part of your install process*. https://github.com/lirantal/npq

[10] Socket. *Introducing Socket Firewall*. https://socket.dev/blog/introducing-socket-firewall

[11] Axios. *Post Mortem: axios npm supply chain compromise*. https://github.com/axios/axios/issues/10636

[12] Microsoft Security Blog. *Mitigating the Axios npm supply chain compromise*. https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/

[13] TanStack. *Postmortem: TanStack npm supply-chain compromise*. https://tanstack.com/blog/npm-supply-chain-compromise-postmortem

[14] TanStack. *Hardening TanStack After the npm Compromise*. https://tanstack.com/blog/incident-followup

[15] GitHub Blog. *Artifact Attestations is generally available*. https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/

[16] GitHub Blog. *Introducing npm package provenance*. https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/

[17] PyPI Blog. *PyPI now supports digital attestations*. https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/

[18] ReversingLabs. *Software Supply Chain Security Report 2026*. https://www.reversinglabs.com/resources/software-supply-chain-security-report-2026